WEB DEVELOPMENT · 3RD SEMESTER

Week 05 · Block 5b

JavaScript: Objects & Modules

<web-development> · Hochschule Ansbach
Matthias Kellnhofer

After this block, you can…

- Use **objects** as both structs and dictionaries
- Grow, shrink, and transform **arrays**
- Explain and write a **closure**
- Write a **class** — and say how it differs from Java
- Split code across **ES modules** with `import` / `export`

Last block: values and functions.

Now: the structures that hold them.

Objects, arrays, closures, classes, and modules — how JavaScript organizes data and code.

PART 1

Objects

Key/value bags

```
const user = {
  name: "Ada",
  age: 36,
};

user.name;      // "Ada"   — dot access
user["age"];    // 36      — bracket access
```

- An object literal `{ }` maps **keys** to **values**
- Read & write with **dot** (`user.name`) or **brackets** (`user["name"]`)
- Values can be anything — including functions (then we call them **methods**)

Flexible dictionaries

```
const scores = {};
scores["ada"] = 10;       // add a key
scores.linus = 7;
delete scores.ada;        // remove a key

const key = "linus";
scores[key];              // 7   — computed key via brackets
```

- Add & remove properties **any time** — no fixed shape
- Brackets take a **variable** as the key — a real dictionary

PART 2

Arrays

Dynamic by nature

```
const xs = [1, 2, 3];
xs.push(4);       // [1, 2, 3, 4]   — grows
xs.pop();         // [1, 2, 3]      — shrinks
xs.length;        // 3
xs[0];            // 1
```

- No fixed size — `push` / `pop` / `shift` / `unshift` grow and shrink it
- Can hold mixed types (though usually you keep them uniform)
- Arrays *are* objects, just keyed by number

Iterate & transform

```
for (const x of xs) console.log(x);

const doubled = xs.map((x) => x * 2);
const evens   = xs.filter((x) => x % 2 === 0);
const total   = xs.reduce((sum, x) => sum + x, 0);
```

- `for...of` walks the **values**
- `map` / `filter` / `reduce` take a **function** — think **Java Streams**
- They return **new** arrays; the callback is a first-class function *(Block 5a!)*

PART 3

Scope & Closures

Scope, briefly

- `let` / `const` live in the **block** `{ }` where they're declared — like Java
- Inner code can **read** variables from the scopes around it
- A function keeps access to **where it was defined** — not where it's called

That last point is what makes closures possible.

Closures

```
function makeCounter() {
  let count = 0;
  return () => ++count;     // remembers `count`
}

const next = makeCounter();
next();   // 1
next();   // 2
```

- A **closure** = a function **+ the variables it closed over**
- `count` survives *after* `makeCounter()` returns — private to `next`
- Exactly how `adder` remembered its number in Block 5a

Why closures matter

- **Private state** — `count` can't be reached or corrupted from outside
- The mechanism behind **callbacks, event handlers, and framework hooks** you'll meet soon
- A quietly powerful idea — you'll reach for it without noticing

PART 4

Classes

Classes look familiar

```
class Account {
  #balance;                          // private field

  constructor(start) { this.#balance = start; }
  deposit(n) { this.#balance += n; }
  get balance() { return this.#balance; }
}

const acc = new Account(100);
acc.deposit(50);
acc.balance;   // 150
```

`class`, `constructor`, methods, `this`, `new`, even `#private` fields — just like Java.

…but they work differently

- **No types** on fields or parameters — it's still dynamic JavaScript
- Under the hood: **prototypes**, not Java-style classes (the `class` keyword is sugar)
- Members are **public by default** (except `#private`); **no overloading**
- Often you don't even need one — an **object literal** or a **closure** does the job

> Reach for a class when you have many instances with shared behaviour. For a one-off bundle of data, an object literal is simpler.

PART 5

Modules

The problem: one big file

- So far all our JS shares **one global scope**
- Real apps span **many files** → name clashes, tangled dependencies
- **Modules**: each file is its **own scope**, and you choose what to share

export / import

```
// math.js
export function add(a, b) { return a + b; }
export const PI = 3.14159;
```

```
// main.js
import { add, PI } from "./math.js";
add(2, 3);   // 5
```

- `export` marks what a file shares; `import` pulls it into another
- **Named** exports (many per file) → import them by name in `{ }`
- Anything not exported stays **private** to its file

default exports & running them

```
// logger.js
export default function log(msg) { /* … */ }

// main.js
import log from "./logger.js";     // any name; no braces
```

- A **default** export: the one "main thing" a file offers
- **Browser**: `<script type="module" src="main.js">`
- **Node**: use `"type": "module"` in `package.json`
- We'll structure real apps exactly this way from **Block 7**

Recap

- **Objects** `{ }` are flexible key/value bags — struct **and** dictionary
- **Arrays** `[ ]` are dynamic; `map` / `filter` / `reduce` take functions *(like Java Streams)*
- A **closure** is a function that remembers its surrounding variables → private state
- **Classes** look like Java but are dynamic **prototype** sugar (with `#private`)
- **Modules** split code into files with their own scope via `export` / `import`

<EXERCISE> · 45 MIN

Your turn 👉

Put JavaScript's structures to work.

1. Tally words with an **object** used as a dictionary
2. Grow an **array** and transform it with `map` / `filter` / `reduce`
3. Build a **closure** — a counter with truly private state
4. Write a **`Stack` class** in its own file, then **`import`** it into `main.js`

Starter: `exercises/week05/5b/` — open `README.md`. `main.js` and a `stack.js` module run with `node`.